Updated game design document — asteroids

Authors

Tim van Naarden
t.vannaarden@students.uu.nl

Matt ter Steege
m.j.tersteege@students.uu.nl

November 6, 2026

Contents

1 Game concept . . . **2**
1.1 Game type and objective . . . 2
2 Core gameplay . . . **2**
2.1 Controls . . . 2
2.2 Rules and progression . . . 2
3 Design . . . **3**
3.1 Data structures . . . 3
3.2 Computer adversary . . . 6
3.3 Interface . . . 6
4 Minimal and optional requirements . . . **7**
4.1 Saving the game state as JSON . . . 7
5 Pure vs impure code . . . **7**
6 Abstractions . . . **8**
6.1 Type classes . . . 8
6.2 Higher-order functions . . . 8
7 Refletion . . . **8**

1 | Game concept

Game type and objective

The game is a two-dimensional arcade game inspired by *Asteroids*. The player pilots a spaceship in an arena that wraps around at the screen edges. Asteroids drift through the arena; the player rotates, accelerates, and fires projectiles to destroy them while avoiding collisions. As the score increases, the game introduces more pressure through additional asteroids and enemy ships with simple pursuit behaviour.

The main objective is to survive for as long as possible and achieve the highest score. The game ends when the player runs out of lives. The visual style will use geometric shapes rendered with Gloss rather than external artwork.

In addition to standard drifting asteroids, hostile scout ships will appear after the player has survived for a while. Scouts steer towards the player's current position, but have a limited turning rate and speed so that their behaviour is predictable and testable. Larger asteroids split into smaller asteroids when destroyed.

2 | Core gameplay

Controls

Input	Action
Left / Right arrows or a / d keys	Rotate the ship
Up arrow or w	Apply thrust in the facing direction
Space	Fire a projectile
P	Pause or resume the game
Enter	Start a new game or restart after game over
F5	Save the current game
F9	Load the saved game

Rules and progression

The ship has a limited number of lives (3). A collision with an asteroid or enemy costs one life and briefly makes the ship invulnerable, preventing removing several lives at once. When an asteroid is hit by a bullet shot by the player, it is removed and awards points. Large and medium asteroids split into smaller ones; the smallest asteroids disappear when destroyed. Enemy ships award more points than asteroids.

The arena uses wrap-around boundaries: an object leaving one edge re-enters on the opposite edge. New asteroids and enemy ships appear at random positions, preferably away from the player to avoid unavoidable collisions. The spawn rate and the number of active threats increase gradually with elapsed play time or score. A game-over screen displays the final score and the locally stored high score.

3 | Design

We follow the model-view-controller structure of the Gloss template. This means that all data types are defined in the `Model` module, the drawing happens in `View`, and the updates happen in `Controller`. The listings in this section are taken directly from our code. `Point` and `Vector` are the Gloss types for a position and a direction, which are both `(Float, Float)`.

Data structures

The game state

The full state of our game is represented using a `GameState` data type:

```
data GameState = GameState {
  gameStatus    :: GameStatus,
  playerData    :: PlayerData,
  asteroids     :: [Asteroid],
  asteroidsWave :: Int,
  highScore     :: Int,
  score         :: Int,
  elapsedTime   :: Float,
  enemies       :: [EnemyData],
  playerBullets :: [Bullet],
  enemyBullets  :: [Bullet],
  explosions    :: [Explosion],
  pressedKeys   :: Set.Set Gloss.Key,
  rngState      :: StdGen
} deriving (Show, Eq)
```

The state contains exactly one `PlayerData`, so the type system already guarantees that there is always a single ship. Asteroids, enemies, bullets, and explosions each have their own list with their own type, instead of one generic list of game objects. As a result, a function that updates asteroids can never receive an enemy by accident. `elapsedTime` is used as the clock of the game. Timers, such as the invulnerability after a death, store the moment they started, which we then compare with `elapsedTime`. `pressedKeys` Keeps track of what keys are press at each frame, so the game can update the player position or other statuses. `rngState` keeps track of the current state of the random module, so each time we need a "random" event, it is at least different from the previous random value. In other words it keeps track of the seed.

The player data

A player is represented as follows:

```
-- | Number of lives the player starts a new game with
startingLives :: Int
startingLives = 3

-- | Seconds the player stays invincible after spawning or dying
invincibilityDuration :: Float
invincibilityDuration = 2

-- | Minimum number of seconds between two shots of the player
fireCooldown :: Float
fireCooldown = 0.25

-- | Maximum speed of the player
playerSpeed :: Float
playerSpeed = 1000

-- | How fast the player speeds up while a movement key is held (units/s^2)
playerAcceleration :: Float
playerAcceleration = 2500

```

```haskell
-- | How fast the ship turns while a turn key is held (degrees/s)
playerTurnSpeed :: Float
playerTurnSpeed = 220

-- | How fast the player slows down when no movement key is held (units/s^2)
playerDeceleration :: Float
playerDeceleration = 1500

-- | Speed of a player bullet (units/s)
bulletSpeed :: Float
bulletSpeed = 800

-- | Seconds a bullet stays alive before it disappears
bulletLifetime :: Float
bulletLifetime = 1

-- | Distance from the ship's centre to its nose, where bullets spawn
playerNoseOffset :: Float
playerNoseOffset = 40

-- | Collision radius of the player's ship
playerRadius :: Float
playerRadius = 25

data PlayerData = PlayerData {
  playerLocation :: Point,
  livesRemaining :: Int,    -- Starts at startingLives
  playerRotation :: Float,  -- Between 0 and 360
  playerVelocity :: Vector, -- Horizontal and vertical speed, thrust adds speed in the
    playerRotation direction
  playerLastDeath :: Float, -- Used for invincibility timer, also grants invincibility on spawn
  playerLastShot :: Float   -- Used for the firing cooldown
} deriving (Show, Eq)
```

We store the rotation and the velocity separately. Thrust adds speed in the direction the ship is facing, but when the ship turns, it keeps drifting in its old direction, just like in the original game. `playerLastDeath` holds the value of `elapsedTime` at the moment of the last death. The player is invulnerable as long as less than `invincibilityDuration` seconds have passed since then. By setting this field to the current time at the start of a game, the player is also protected right after spawning.

`playerLastShot` works in the same way for shooting. A new bullet is only fired when at least `fireCooldown` seconds have passed since the last shot. Without this cooldown, holding the space bar would fire a bullet every frame, which would make the game far too easy.

The enemies

Enemies are represented using the following data type:

```haskell
data EnemyData = EnemyData {
  enemyLocation :: Point,
  enemyRotation :: Float, -- Between 0 and 360
  enemyVelocity :: Vector -- Horizontal and vertical speed, the enemy steers it towards the player
} deriving (Show, Eq)
```

They can shoot bullets at the player's current location. How they decide in which direction to move is described in the computer adversary section (Section 3.2). Enemies have their own set of global variables that set the radius, speed, firecooldown, bulletspeed and -lifetime, aim error, spawn interval and max amount of enemies at each given time.

Bullets

Bullets can be fired either by the player or by one of the enemies:

```haskell

```

```haskell
data Bullet = Bullet {
  bulletLocation :: Point,
  bulletVelocity :: Vector, -- Bullets have no rotation (just a dot), so this is the horizontal and
    vertical speed
  bulletAge :: Float
} deriving (Show, Eq)
```

All bullets are stored in two lists. A `PlayerBullet` destroys asteroids and enemies, while an `EnemyBullet` can only hit the player. This prevents enemies from destroying each other or the asteroids. The bulletAge determines for how long the bullet exists.

The asteroids

When an asteroid is hit, it splits into two smaller asteroids. Only the smallest asteroids disappear completely.

```haskell
data AsteroidSize = Small | Medium | Large
  deriving (Enum, Show, Eq, Bounded, Generic, ToJSON, FromJSON)

data Asteroid = Asteroid {
  asteroidLocation :: Point,
  asteroidSize :: AsteroidSize,
  asteroidVelocity :: Vector,
  asteroidVertices :: [Vector],
  asteroidRotation :: Float
} deriving (Show, Eq, Generic, ToJSON, FromJSON)
```

Because `AsteroidSize` derives `Enum` and `Bounded`, we do not need a separate case for every size when splitting. A hit asteroid is replaced by two asteroids of the previous size (using `pred`), and an asteroid of size `minBound` is removed. The size also determines the radius that is used for drawing and collision checks, and the number of points the player receives.

```haskell
splitAsteroid  :: Asteroid → [Asteroid]
asteroidRadius :: AsteroidSize → Float
asteroidPoints :: AsteroidSize → Int
```

Furthermore, it also keeps track of it's own vertices, so that we can draw the rugged edges the same each frame. That is why we also keep track of rotation, so the vertices are not at the same absolute angle.

When two asteroids collide with each other, they are bounced back at 180 degrees from their original travel path. This may not be the most realistic, but it was an easier implementation that being physically correct.

Explosions

Whenever an enemy is destroyed, an explosion is added to the game state:

```haskell
data Explosion = Explosion {
  explosionLocation :: Point,
  explosionVelocity :: Vector,
  explosionAge :: Float,      -- Seconds since the explosion started
  explosionDuration :: Float -- The explosion is removed once its age reaches this
} deriving (Show, Eq)
```

In contrast to the timers of the player, an explosion stores its own age instead of a start time. The reason for this is that the drawing of an explosion depends on how far along it is. For example, the radius of the burst can be computed directly from `explosionAge / explosionDuration`.

Game status

The game has several screens, and the game should only update while it is actually being played. Therefore, the current screen is stored in the game state as a `GameStatus`:

```haskell
data GameStatus = Title | Playing | Paused | GameOver deriving (Show, Eq)
```

Both the update and the input functions first look at `gameStatus`.

Moving objects

The player, enemies, asteroids, and bullets all have a location and a velocity, and they all move in the same way every update. Instead of writing a separate move function for each type, we let them share the type class `Movable`:

```
-- | Everything that drifts through the arena with a velocity
class Movable a where
  location    :: a → Point
  velocity    :: a → Vector
  setLocation :: Point → a → a

-- | Move an object along its velocity for the given number of seconds
move :: Movable a ⇒ Float → a → a
move secs x = setLocation (px + vx * secs, py + vy * secs) x
  where (px, py) = location x
        (vx, vy) = velocity x

instance Movable Asteroid where
  location        = asteroidLocation
  velocity        = asteroidVelocity
  setLocation p x = x { asteroidLocation = p }
```

The instances for `PlayerData`, `EnemyData`, and `Bullet` follow the same pattern. This means that `move` is only defined once, and updating all asteroids becomes `map (move secs) (asteroids gstate)`. Wrapping around the screen edges and checking for collisions only depend on the location as well. Therefore, these functions will also be written once for every `Movable`:

```
wrapAround :: Movable a ⇒ a → a
collides   :: (Movable a, Movable b) ⇒ Float → a → b → Bool
```

Computer adversary

After the player has survived for a while, scout ships will start to appear (each 15 seconds). Each update, a scout computes the angle from its own location to the player and turns `enemyRotation` towards it. However, it can never turn faster than a fixed turning rate. Its velocity is then accelerated in the new direction, up to a maximum speed. As a result, scouts chase the player, but they are never impossible to escape. When the player is roughly in front of a scout, it fires an `EnemyBullet`.

```
updateEnemy :: Float → PlayerData → EnemyData → EnemyData
enemyShoot  :: PlayerData → EnemyData → Maybe Bullet
```

Both functions are pure and only depend on the player and the enemy itself.

Interface

The game is drawn with Gloss using `playIO`. Gloss requires the view function to work in IO, even though drawing does not need any side effects. Therefore, the view function only calls a pure function that builds the picture:

```
view     :: GameState → IO Picture
viewPure :: GameState → Picture
```

`viewPure` combines a picture for every element of the game state using `pictures`. The ship is drawn as a triangle rotated by `playerRotation`, asteroids as circles with a radius based on their size, and bullets as small dots. Each of these types will be an instance of the `Drawable` class, which is described in the abstractions section (Section 6). On top of the arena, the score, the high score, and the remaining lives are shown as text. The `Title`, `Paused`, and `GameOver` statuses each get their own overlay.

```
drawHud    :: GameState → Picture
drawScreen :: GameStatus → Picture
```

Requirement	Implemented
Keyboard-controlled player	Yes, keybinds are on startscreen
Enemies with basic intelligence	Yes, they shoot a bullet to the players location with a certain error
Randomness	Yes, via build-in library
Multi-frame animation	Rocket flame, alien saucer lights, explosions
Pause	Yes, by pressing P to toggle pause/play
File-system interaction	Yes, The high score is saved and loaded from a file across restarts
Save to JSON	Yes, save/load gamestate using JSON files (★ optional)

Keyboard input is handled in the same way. The IO version required by `playIO` calls a pure function that translates a key event into a change of the game state, following the controls in the table above.

```
input    :: Event → GameState → IO GameState
inputKey :: Event → GameState → GameState
```

4 | Minimal and optional requirements

Saving the game state as JSON

Pressing F5 will save the full game state to a file, and pressing F9 will load it again. We will store the state as JSON using the Aeson library. Instead of writing the conversion by hand, every type in the game state will derive `Generic, ToJSON and FromJSON`. After that, Aeson can generate the JSON for us

```
data GameState = GameState { ... } deriving (Show, Eq, Generic, ToJSON,  FromJSON)
```

The same is done for `PlayerData`, `EnemyData`, `Asteroid`, `AsteroidSize`, `Bullet`, `BulletType`, `Explosion`, `GameStatus`. `Point` and `Vector` are tuples of floats, which Aeson already supports.

When the file is missing or cannot be decoded, `loadGame` returns `Nothing` and the current game simply continues. A loaded game starts with the status `Paused`, so that the player has a moment to see the situation before playing on. The high score remains a separate plain text file, so it is not lost when a save file is overwritten.

5 | Pure vs impure code

Gloss expects the step, input, and view functions to work in IO. However, most of the game does not need any side effects. Therefore, the main step loop, will call functions using a

```
    State StdGen a
```

wrapper. However, this is only needed for the functions that need to generate random nummers.

```
step     :: Float → GameState → IO GameState
stepPure :: Float → GameState → GameState

input    :: Event → GameState → IO GameState
inputKey :: Event → GameState → GameState

view     :: GameState → IO Picture
viewPure :: GameState → Picture
```

The impure parts of the game are limited to the following:
1. Reading the high score at startup in `main`, and writing it in `step` when a new high score is reached.
2. Saving and loading the game state when F5 or F9 is pressed.
3. Choosing the first `Seed` once at startup, for example based on the current time.

Because the random generator is stored explicitly in the game state, spawning asteroids and enemies is pure as well. A spawn function receives a `Seed` and returns the new objects together with the next `Seed`. Movement,

collisions, splitting asteroids, the steering of enemies, animations, and drawing are all pure functions of the game state.

6 | Abstractions

Type classes

The `Movable` class (Section 3.1.8) removes the duplication between the four moving types. This means that `move`, `wrapAround`, and `collides` are each written once instead of four times.

Higher-order functions

Since every kind of object is stored in a list, most updates can be written with higher-order functions instead of explicit recursion. For example:
1. Moving all objects is a `map` over each list.
2. Destroyed objects and finished explosions are removed with `filter`.

In addition, constants such as `startingLives`, `invincibilityDuration`, and `fireCooldown` are defined once with a name. Therefore, no magic numbers appear in the update functions.

7 | Refletion

Although the teamwork between us was close, we still found ourselves struggling with the last pieces of work on the day of the deadline. A good lesson to take from this is that the next time we work together, we have to create strict deadlines for multiple substeps of the project. This way, we can plan the finishing date a couple of days before the actual deadline. This reduces deadline-day stress and will result in cleaner work.